Fifteen Runs to Beat a Baseline: What Hyperparameter Tuning Taught Me About Overfitting and Early Stopping in a Keras Sentiment Classifier

Luke Joaquin Bernardo
College of Computer Studies and Engineering
Jose Rizal University
Taguig City, Philippines
lukejoaquin.bernardo@my.jru.edu

***Abstract*—This report covers what I found while rebuilding my PT-P1 no-code sentiment classifier in Keras. I trained a small neural network on the same 332 sport e-commerce customer messages, sorted into Positive Praise, Negative Complaint, and Urgent Support, and ran fifteen separate experiments across four tasks: a default baseline, three runs meant to force underfitting, overfitting, and regularization, and ten one-variable-at-a-time ablations on top of that. My PT-P1 benchmark, after refinement, sat at 43.3 percent validation accuracy on ambiguous phrasing. Every Keras run I tried beat that, and my strongest configuration, which combined a larger dataset, early stopping, a bigger vocabulary, and a smaller batch size, reached a weighted F1 score of 0.8416. Along the way I saw clear underfitting, clear overfitting, and the exact point where early stopping saves a model from itself. I use these results to talk through where explicit hyperparameter control actually helps compared to no-code automation, how dropout and model size interact with overfitting, why more data beat more capacity in my case, and where in the code the forward pass, loss calculation, and backward pass actually happen.**

I. Introduction and PT-P1 Baseline Performance

For PT-P1 I built a no-code text classifier using Machine Learning for Kids, trained on 332 customer messages from a sports team e-commerce and ticketing platform, split evenly across three classes. On clear, everyday phrasing the model reached 90.0 percent accuracy. On sarcastic or ambiguous phrasing it dropped to 33.3 percent, and after I added 32 refinement examples targeting the failure patterns I saw, it climbed back to 43.3 percent. That 43.3 percent number is the benchmark this report tries to beat, since it reflects the final, refined version of the no-code model rather than its first attempt.

The goal of this exercise was to take that same 332 row dataset into a Keras notebook and see whether directly controlling the architecture, the optimizer, and the regularization could do better than the no-code tool's built in heuristics. I ran fifteen experiments in total. The first five follow the assignment's required progression: a baseline, then three runs that deliberately push the model toward underfitting, overfitting, and a regularized middle ground, then one hand-picked custom configuration. The remaining ten are a free-form ablation chain, where I started from whichever combination worked best so far and changed exactly one setting at a time, so I could tell which change actually helped and which one did not.

II. Experimental Methodology and Hyperparameter Configurations

The pipeline itself stays the same across every run. I load the 332 row CSV, lowercase and split each message into words, and build a vocabulary out of the most common words, reserving one slot for padding and one for anything outside the vocabulary. Every message gets turned into a fixed length sequence of twenty word indices, padding short messages with zeros and cutting off long ones. I split the data 70 percent train, 15 percent validation, and 15 percent test, and I never touch the test set until the very end, so nothing about hyperparameter tuning can leak into it.

The model architecture follows the same four layer shape in every run. An embedding layer turns each word index into a learned vector. A global average pooling layer then averages those vectors across the sentence into one fixed length representation, which is the actual feature extraction step, since a neural network cannot do math on raw words. That representation goes through one dense hidden layer with a ReLU activation, then a dropout layer, then a final dense layer with a softmax activation that outputs a probability for each of the three classes. I think of the hyperparameters in three groups, following the lecture's three pillars. Architecture covers the embedding size and the hidden layer size. Optimization covers the learning rate, the optimizer choice, the batch size, and the number of epochs. Regularization covers the dropout rate.

Table I lists every run with its key settings and its final validation metrics. Runs EXP-01 through EXP-05 follow Task 2's required progression. Runs EXP-05A through EXP-05J are the Task 4 ablation chain, where EXP-05C combines the two things that helped on their own, early stopping and a larger

dataset, and every run after that changes exactly one more setting off that shared base.

EXP-01 through EXP-05 train on the original 332 example dataset. EXP-05B onward trains on an expanded 452 example dataset, built by combining the original data with 120 new hand written examples while keeping the classes close to balanced, at 151, 151, and 150 examples per class. EXP-05I is the one exception in that range, since its whole point was to isolate the effect of a bigger hidden layer from the effect of more data, so it deliberately goes back to the original 332 example set.

III. Performance Monitoring and Loss Curve Analysis

Every run's loss curve plots training loss against validation loss per epoch, which is the fastest way to see whether a model is underfitting, overfitting, or landing somewhere reasonable in between. Figure 1 shows the five required runs from Tasks 1 and 2, and Figures 2 and 3 show the ten ablation runs from Task 4.

EXP-01, the plain baseline, tells a clear overfitting story on its own. Training loss falls from about 1.09 to basically zero by epoch fifteen, while validation loss bottoms out at 0.4263 around epoch seven and then climbs back to 0.6684 by the end. That gap between a training accuracy of 100 percent and a validation accuracy of 78 percent is the model memorizing the training examples once it runs out of new patterns to learn. EXP-02 looks completely different. Both curves stay high and close together the whole time, training loss barely drops below 1.09, and validation accuracy caps out at 56 percent. That is underfitting exactly as intended, since a hidden layer of 16 units and a learning rate of 0.0001 barely gives the model enough room or enough push to learn anything past the most obvious patterns. EXP-03 is the most extreme curve of the whole set. Training loss collapses to almost exactly zero by the end while validation loss more than doubles from where it started, reaching 2.2235. A hidden layer of 256 units, a learning rate of 0.05, and zero dropout gives the model every tool it needs to memorize the training set and no reason not to. EXP-04 adds dropout of 0.4 on top of that same high capacity, and the validation loss curve rises more gently than EXP-03's does, bottoming around 0.4913 before climbing to 0.6551, which shows dropout tempering the overfitting without fully stopping it. EXP-05, the hand picked custom run, lands close to EXP-01 and EXP-04 in final numbers but gets there with a slightly lower validation loss overall, which is really just a preview of what a better tuned configuration can do before the more systematic Task 4 runs.

EXP-05A isolates early stopping on its own, and the curve makes the payoff obvious. Training runs for twenty epochs of headroom but stops at epoch nine, since validation loss stopped improving after epoch six, and the callback rolls the weights back to that best point, giving a final F1 of 0.7967 against EXP-01's 0.7836 with the exact same architecture. EXP-05B isolates the expanded dataset on its own, no early stopping this time, and validation loss still climbs back up by the end since nothing is there to catch it, but the F1 still improves to 0.8095 because the model has more to learn from before it starts memorizing. EXP-05C stacks both changes together, and the result is the best of any single lever alone, an F1 of 0.8377, reached in only eight epochs before early stopping kicks in. That combined run becomes the shared starting point for everything from EXP-05D onward.

From there the ablation chain gets more mixed. EXP-05D raises the hidden layer from 64 to 128 on top of EXP-05C and actually comes out slightly worse, an F1 of 0.7955, which tells me the extra capacity was not the bottleneck once early stopping and more data were already in place. EXP-05E raises dropout from 0.1 to 0.3 and lands almost exactly where EXP-05C did, an F1 of 0.8376, so extra regularization did not add much once overfitting was already being controlled by early stopping. EXP-05F swaps the flat learning rate for a cosine decay schedule and edges up to 0.8390, the third best run overall, which lines up with the lecture's point that schedules can smooth out training near the end. EXP-05G rebuilds the vocabulary with a higher cap after I noticed the expanded dataset actually has 1,377 unique words while the original 1,000 word cap was silently collapsing about 27 percent of them into an unknown token, and that recovered vocabulary pushes F1 to 0.8407, the second best of the whole study. EXP-05H is the one clear failure in this chain. It keeps every setting from EXP-05C except swapping Adam for plain SGD at the same learning rate, and training barely gets off the ground, stopping after only four epochs with an F1 of 0.4184, close to random guessing across three classes. EXP-05I goes back to the original, non-expanded dataset with EXP-05D's larger hidden layer, specifically to check whether the capacity increase interacts differently without the extra data, and it actually scores higher than EXP-05D did, an F1 of 0.8205, though its validation set is smaller so I read that comparison as suggestive rather than conclusive. EXP-05J drops the batch size from 32 to 16 on top of EXP-05C and produces the best result of the entire study, an F1 of 0.8416, though it sits close enough to EXP-05F and EXP-05G that I treat all three as a top cluster rather than one clear winner.

IV. Discussion and Debrief Answers

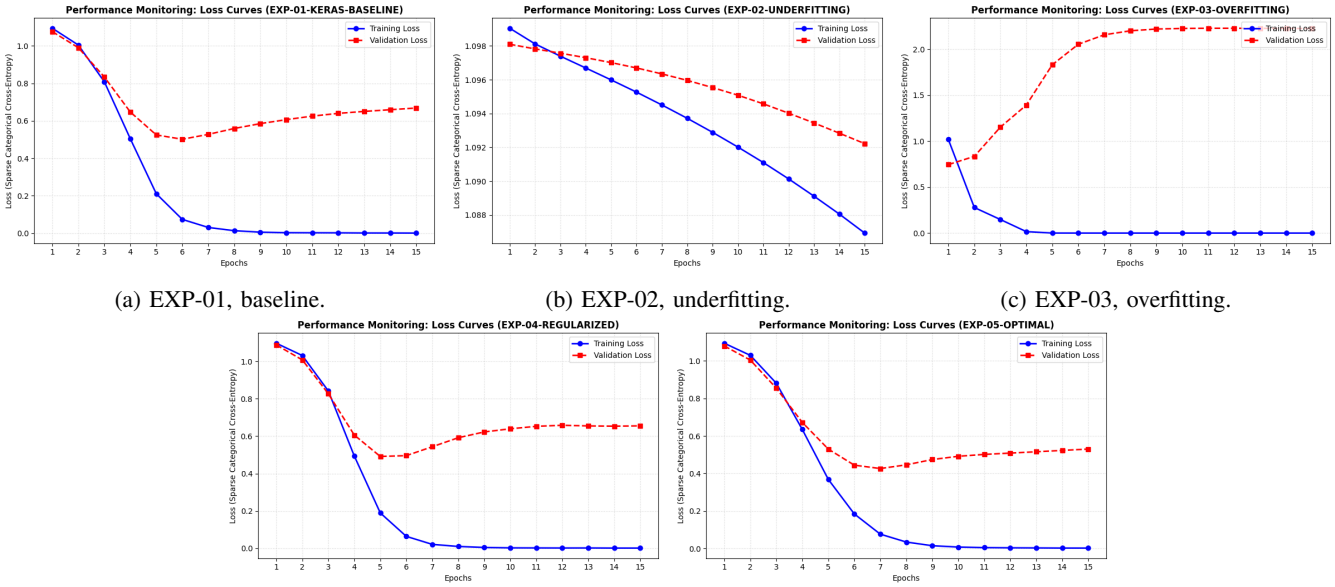
A. Code Based Optimization versus No Code Heuristics

Working directly in Keras made the trade offs between explicit control and automation a lot more concrete than PT-P1 did. In the no-code tool, every hyperparameter decision was hidden behind one training button, so when the model misread sarcasm I had no way to ask whether that was a learning rate problem, a capacity problem, or something about how the tool extracts features from text. In Keras, EXP-05H answered a question I could never have even asked in the no-code tool: is a learning rate of 0.005 doing the same thing regardless of which optimizer uses it? It is not. Adam adapts its effective step size per parameter, so 0.005 works fine for it, but the exact same number handed to plain SGD is nowhere near enough movement, and the model barely trains before early stopping

TABLE I: Summary of all fifteen training runs, their key hyperparameters, and final validation metrics.

Run ID	Hidden	LR	Optimizer	Epochs	Batch	Dropout	Val Loss	Val Acc (%)	Val F1
EXP-01-KERAS-BASELINE	64	0.005	Adam	15/15	32	0.1	0.6684	78.00	0.7836
EXP-02-UNDERFITTING	16	0.0001	Adam	15/15	32	0.0	1.0922	56.00	0.4579
EXP-03-OVERFITTING	256	0.05	Adam	15/15	32	0.0	2.2235	72.00	0.7090
EXP-04-REGULARIZED	128	0.005	Adam	15/15	32	0.4	0.6551	78.00	0.7836
EXP-05-OPTIMAL	128	0.003	Adam	15/15	32	0.3	0.5301	78.00	0.7841
EXP-05A-EARLY-STOPPING	64	0.005	Adam	9/20	32	0.1	0.4669	80.00	0.7967
EXP-05B-EXPANDED-DATA	64	0.005	Adam	15/15	32	0.1	0.6301	80.88	0.8095
EXP-05C-COMBINED	64	0.005	Adam	8/20	32	0.1	0.4517	83.82	0.8377
EXP-05D-HIGHER-CAPACITY	128	0.005	Adam	8/20	32	0.1	0.4590	79.41	0.7955
EXP-05E-MORE-DROPOUT	64	0.005	Adam	9/20	32	0.3	0.4426	83.82	0.8376
EXP-05F-LR-SCHEDULE	64	0.005*	Adam	9/20	32	0.1	0.4388	83.82	0.8390
EXP-05G-LARGER-VOCAB	64	0.005	Adam	8/20	32	0.1	0.4288	83.82	0.8407
EXP-05H-SGD-OPTIMIZER	64	0.005	SGD	4/20	32	0.1	1.0968	42.65	0.4184
EXP-05I-CAPACITY-ONLY	128	0.005	Adam	8/20	32	0.1	0.4336	82.00	0.8205
EXP-05J-BATCH-SIZE	64	0.005	Adam	7/20	16	0.1	0.4539	83.82	**0.8416**

*Initial learning rate under a cosine decay schedule, not a flat value. Epochs shown as trained/cap for runs using early stopping.



(a) EXP-01, baseline. (b) EXP-02, underfitting. (c) EXP-03, overfitting.

(d) EXP-04, regularized. (e) EXP-05, custom.

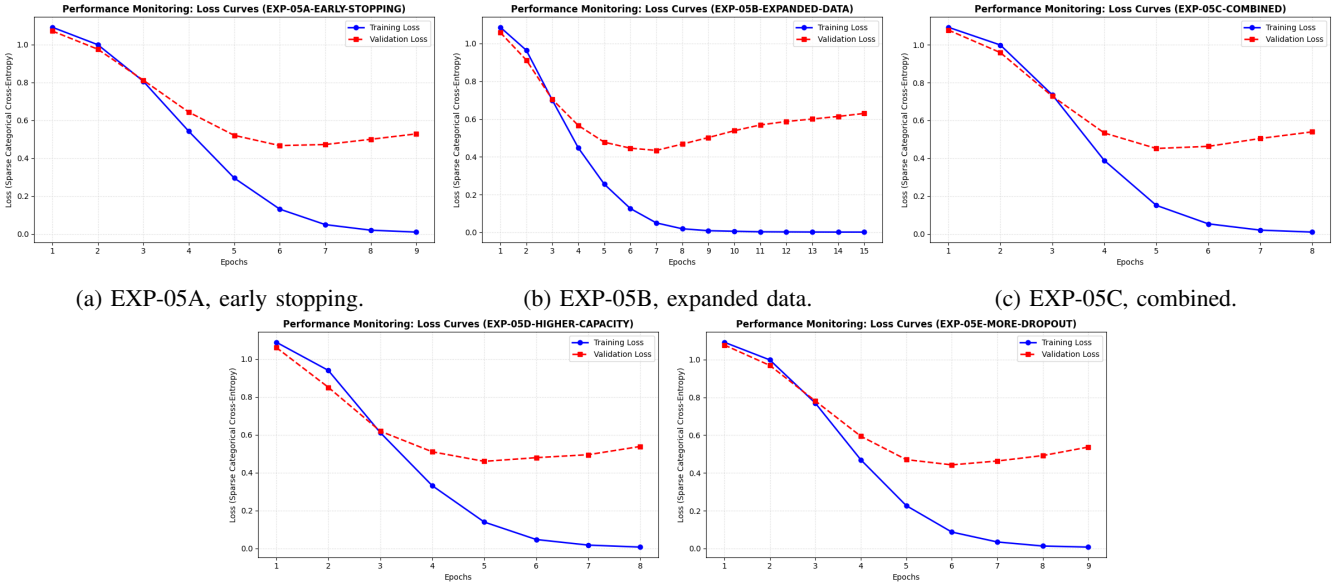
Fig. 1: Loss curves for the five Task 1 and Task 2 runs. EXP-02 stays flat and high on both curves, EXP-03 shows training loss collapsing toward zero while validation loss climbs, and EXP-01, EXP-04, and EXP-05 all bottom out around epoch six or seven before drifting back up.

gives up on it. That is a distinction a no-code tool simply does not expose.

The cost of that control is that every choice becomes my responsibility instead of the tool's. Picking the wrong learning rate, the wrong batch size, or the wrong dropout rate can quietly tank a run, and I only find out by comparing loss curves after the fact, which is exactly why I leaned on one-variable-at-a-time ablations for Task 4 rather than changing several things per run. No-code automation trades that flexibility for speed and safety. It picks reasonable defaults so a beginner cannot easily break training, but it also means the person using it cannot diagnose why a model succeeds or fails past a surface level. My PT-P1 report noted that the no-code tool's failures on sarcasm probably came from a bag of words style feature extraction. In Keras I can actually confirm that kind of thing by reading the code, which is not something the no-code interface lets you do.

B. Diagnosing Model Fit through Loss Curves and Regularization

Across all fifteen runs, the underfitting and overfitting patterns showed up exactly the way the lecture describes them. Underfitting, in EXP-02, looked like both curves staying high and flat together, since a hidden layer of 16 units and a learning rate of 0.0001 never gave the model enough room or enough push to fit the data at all. Overfitting, most clearly in EXP-03, looked like training loss collapsing toward zero while validation loss climbed the whole time, which is the model



(a) EXP-05A, early stopping. (b) EXP-05B, expanded data. (c) EXP-05C, combined.

(d) EXP-05D, higher capacity. (e) EXP-05E, more dropout.

Fig. 2: Loss curves for the first half of the Task 4 ablation chain. EXP-05C stacks early stopping and expanded data on top of each other, and EXP-05D and EXP-05E each change one more thing off that shared base.

memorizing specific training examples instead of learning anything that generalizes.

Dropout and hidden layer size interact with that pattern in ways that were not always what I expected going in. Raising dropout from 0.1 to 0.4 in EXP-04, while keeping the same high capacity as EXP-03, softened the overfitting noticeably. Validation loss's minimum landed around 0.4913 instead of climbing straight through 2.2235, so dropout is doing real work there. But later in the ablation chain, EXP-05E raised dropout from 0.1 to 0.3 on top of a configuration that already had early stopping and more data, and it barely changed anything, an F1 of 0.8376 against EXP-05C's 0.8377. My read on that is dropout mostly matters when overfitting is actually the main problem left to fix. Once early stopping is already cutting training off near the validation loss minimum, there is less overfitting left for dropout to prevent, so adding more of it does not have much left to do. Hidden layer size told a similar story. EXP-05D raised `hidden_dim` from 64 to 128 on top of EXP-05C and came out slightly worse, which suggests that once the model already has enough capacity to fit the patterns in the data, adding more capacity does not help and can even hurt slightly, probably by giving the model more room to latch onto noise before early stopping catches it.

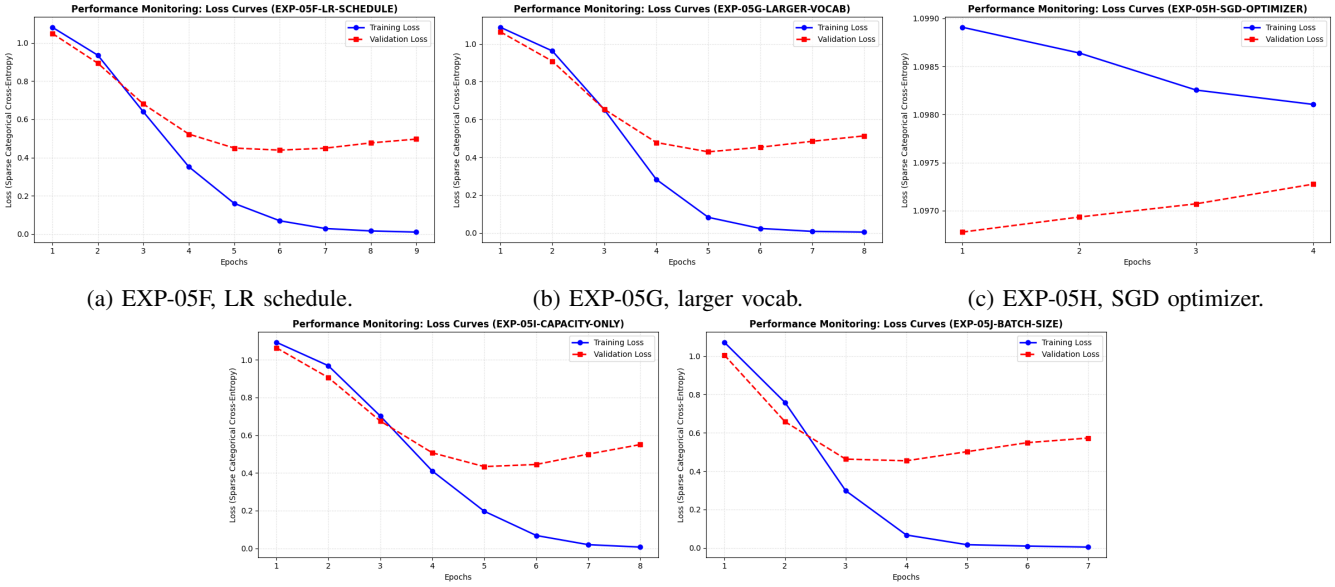
C. Data Volume and Diversity versus Architectural Capacity

Hyperparameter tuning alone, in EXP-01 through EXP-05, already cleared the 43.3 percent PT-P1 benchmark by a wide margin, so I did not strictly need to expand the dataset to beat that number. But I still ran the expansion in Task 4 because the ablation chain needed a stronger baseline to test the rest of the levers against, and the results back up why that was worth doing. EXP-05B, more data with no other changes, pushed F1 from EXP-01's 0.7836 to 0.8095. EXP-05C, that same expanded data combined with early stopping, pushed it further to 0.8377. That is a real gain from more data, not just noise.

The more interesting comparison is EXP-05D against EXP-05I, since those two isolate capacity from data in opposite directions. EXP-05D uses the larger 128 unit hidden layer on the expanded 452 example dataset and scores an F1 of 0.7955. EXP-05I uses that exact same 128 unit hidden layer but on the original 332 example dataset and scores 0.8205, actually higher, though with a smaller validation set that makes the comparison less certain than I would like. Taken at face value, that tells me capacity was not the bottleneck in this study. Adding more data helped every time I tried it, but adding more capacity on top of data that was already sufficient did not, and in EXP-05D's case might have even worked against the model slightly. That lines up with what I found in PT-P1, where volume without the right kind of diversity did not fix the sarcasm blind spot. Here, the diversity came from writing genuinely new example sentences rather than duplicating existing patterns, which is probably why the data expansion helped as consistently as it did.

D. Connecting Code Implementation to Slide Architecture

The forward pass, loss calculation, backward pass, and weight update all happen inside a single call in this notebook, `model.fit(X_train, y_train, ...)`, on line 24 of the optimizer and training cell for each experiment, for example the EXP-01 cell. Keras hides the mechanics of that loop behind the `.fit()` call, but all four steps from the lecture's training loop diagram are still happening underneath it, once per mini batch, for every epoch. TensorFlow's automatic differentiation is what makes the backward pass



(a) EXP-05F, LR schedule. (b) EXP-05G, larger vocab. (c) EXP-05H, SGD optimizer.

(d) EXP-05I, capacity only. (e) EXP-05J, smaller batch, champion model.

Fig. 3: Loss curves for the second half of the Task 4 ablation chain. EXP-05H shows the clearest failure of the whole study, and EXP-05J produced the best weighted F1 score of all fifteen runs.

possible without me writing gradient calculations by hand, tracking every operation from the forward pass so it can compute the loss gradient with respect to every weight through the chain rule, then handing those gradients to the optimizer object built two lines earlier, on lines 4 through 7, which performs the actual weight update.

Feature extraction happens earlier and separately from all of that, in the data loading cell, specifically lines 32 and 33, where `build_vocab` and `text_to_sequence` turn raw sentences into fixed length integer sequences. That step alone is not really extracting features in the way the embedding layer does, since it is just assigning arbitrary integer IDs to words. The real automated feature extraction happens once those integer sequences enter the model. The embedding layer, defined on lines 34 through 38 of the architecture cell, turns each integer into a learned vector, and training adjusts those vectors so that words used in similar ways end up with similar vectors, which is the mathematical association the lecture describes replacing hand written IF THEN rules. The global average pooling layer right after it, on line 41, then compresses that whole sequence of word vectors into one fixed length vector per message by averaging across the sentence, which is what lets a variable length sentence turn into a fixed size input for the dense layers that follow. So in terms of the pipeline, tokenizing text into integers is bookkeeping, but the embedding and pooling layers together are where the actual feature extraction the lecture describes takes place, and everything after that, the two dense layers and the softmax output, is where the network turns those extracted features into a classification.

V. Conclusion

Rebuilding the PT-P1 classifier in Keras made the tradeoff between automation and control very concrete. Every one of my fifteen runs beat the 43.3 percent no-code benchmark, and the best of them, EXP-05J at an F1 of 0.8416, got there by stacking early stopping, an expanded and more diverse dataset, a recovered vocabulary, and a smaller batch size on top of each other rather than any single change doing all the work. I saw textbook underfitting in EXP-02, textbook overfitting in EXP-03, and a genuinely useful lesson in EXP-05H, where the exact same learning rate that worked fine for Adam nearly broke training under plain SGD, something a no-code tool would never have let me discover in the first place. The capacity ablations in EXP-05D and EXP-05I also pushed back on my first instinct, which was that a bigger model should do better. Once early stopping and more data were already in place, extra capacity did not help and sometimes hurt slightly. If I take one lesson from this whole exercise, it is that hyperparameter tuning is not really about finding one magic setting. It is about testing one change at a time, reading the loss curves honestly, and being willing to find out that the change you expected to help did not.